

# WebLab KTV Airline - FileUpload Vulnerability Flow

Tài liệu này ghi lại flow phát hiện lỗ hổng **FileUpload: blacklist bypass + path traversal** trong repo `WebLab_KTVAirline`.  
Cách trình bày ưu tiên import vào Notion: heading rõ, bảng summary, checklist, callout, code block có chú thích ngay cạnh sink/source.

## 1. Phạm vi rà soát

| Hạng mục                                                 | Trạng thái          | Điểm đánh dấu đã đọc                                                                                                                                                 |
|----------------------------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FileUpload: blacklist bypass, path traversal             | Đã rà soát chi tiết | <code>UserController.uploadAvatar</code> , <code>UserServiceImpl.uploadAvatar</code> , <code>WebConfig</code> , <code>/uploads/**</code>                             |
| XXE / XMLDecoder                                         | Đã rà soát          | <code>PromotionController.importPromotionXml</code> , <code>PromotionServiceImpl.queuePromotionXml</code> , <code>PromotionServiceImpl.importPromotionXml</code>     |
| SQLi error/boolean                                       | Đã rà soát          | <code>FlightSearchRepositoryImpl.appendKeywordSearch</code> , <code>TransactionSearchRepositoryImpl.appendFlightNameSearch</code>                                    |
| JWT header <code>jwk</code> / <code>kid</code> injection | Đã rà soát          | <code>JwtUtils.resolveVerificationKey</code> , <code>JwtUtils.resolveKeyFromKidCommand</code>                                                                        |
| JWT algorithm confusion                                  | Đã rà soát          | <code>JwtUtils.init</code> , <code>JwtUtils.resolveVerificationKey</code> , <code>JwtUtilsTest.acceptsHs256TokenSignedWithPublicKeyPemAsHmacSecret</code>            |
| Command Injection blind                                  | Đã rà soát          | <code>QRCodeHelper.renderQrCode</code> , <code>Runtime.getRuntime().exec("/bin/sh", "-c", command)</code>                                                            |
| SSTI object-chain + dev-supplied template                | Đã rà soát          | <code>ProfileViewController</code> , <code>FreeMarkerSandboxConfig</code> , <code>CustomThemeLoader</code> , <code>ProfileTheme</code> , <code>light_mode.ftl</code> |
| LFI log poisoning                                        | Đã rà soát          | <code>application.properties</code> access log config, <code>CustomThemeLoader.loadFilesystemTheme</code> , <code>renderTemplateSource</code>                        |
| Insecure Deserialize                                     | Đã rà soát          | <code>BookingController.importDraft</code> , <code>BookingServiceImpl.importDraft</code> , <code>ObjectInputStream.readObject</code>                                 |
| Race Condition: limit overrun                            | Đã rà soát          | <code>BookingServiceImpl.applyPromotion</code> , <code>BookingOrderRepository.applyPromotionDiscount</code> , <code>BookingOrder.appliedPromotionCodes</code>        |

## 2. Kết luận nhanh cho FileUpload

| Thuộc tính          | Giá trị                                                                                                                        |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Entry point         | POST /api/v1/user/{id}/avatar                                                                                                  |
| Quyền truy cập      | ROLE_USER hoặc ROLE_ADMIN, và user chỉ upload cho chính mình nếu không phải admin                                              |
| Source chính        | MultipartFile file, optional multipart field filename, MultipartFile#getOriginalFilename()                                     |
| Sink chính          | file.transferTo(destination.toFile())                                                                                          |
| Stored path         | uploads/avatars/<filename>                                                                                                     |
| Public exposure     | /uploads/** được permit và map thẳng tới app.upload-dir                                                                        |
| Lỗi blacklist       | Chỉ chặn đuôi file bằng filename.endsWith(...), case-sensitive, không whitelist MIME/extension ảnh                             |
| Lỗi path traversal  | Dùng regex yếu để chặn ../, sau đó avatarDir.resolve(filename) mà không normalize() + startsWith/avatarDir) trước khi ghi file |
| Fix comment hiện có | UserServiceImpl.java dòng 127-148                                                                                              |

**Nhận định:** đây là vulnerable flow hoàn chỉnh: attacker điều khiển filename/file content, backend dùng filename đó để build path ghi file, blacklist không đủ mạnh, traversal filter có bypass, file được ghi vào thư mục public hoặc thoát khỏi avatars.

## 3. Sơ đồ dữ liệu tổng quát

Browser Profile.jsx

```
-> uploadUserAvatar(user.id, file)
-> POST /api/v1/user/{id}/avatar multipart/form-data
-> UserController.uploadAvatar(...)
-> UserServiceImpl.uploadAvatar(id, file, filename)
-> filename = requested filename OR file.getOriginalFilename()
-> rejectRelativeParentPath(filename) [weak traversal filter]
-> isBlockedByBlacklist(filename) [blacklist bypass]
-> destination = avatarDir.resolve(filename) [path controlled by filename]
-> file.transferTo(destination.toFile()) [file write sink]
```

```
-> user.avatarUrl = "/uploads/avatars/" + filename
-> /uploads/** static handler serves app.upload-dir
```

## 4. Hướng 1 - Truy vết từ sink đến source

### 4.1. Bắt đầu từ sink ghi file

**File:** `src/main/java/org/example/serviceImpl/UserServiceImpl.java`

```
Path avatarDir = Paths.get(uploadDir,
"avatars").toAbsolutePath().normalize();
Files.createDirectories(avatarDir);

String filename = resolveUploadFilename(file, requestedFilename);

rejectRelativeParentPath(filename, "Filename cannot contain ../");
if (isBlockedByBlacklist(filename)) {
    throw new IllegalArgumentException("This file type is not allowed");
}

Path destination = avatarDir.resolve(filename);           // [SINK-ADJACENT]
if (destination.getParent() != null) {
    Files.createDirectories(destination.getParent());      // [PATH WRITE PREP]
}

file.transferTo(destination.toFile());                    // [SINK]
user.setAvatarUrl("/uploads/avatars/" + filename);        // [STORED OUTPUT]
```

| Dòng phân tích                           | Ý nghĩa                                                                                                                |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <code>avatarDir</code>                   | Base directory hợp lệ được normalize: <code>uploads/avatars</code> hoặc <code>/app/uploads/avatars</code> trong Docker |
| <code>filename</code>                    | Không phải server-generated; đến từ request hoặc <code>OriginalFilename</code>                                         |
| <code>rejectRelativeParentPath</code>    | Có kiểm tra traversal nhưng regex yếu                                                                                  |
| <code>isBlockedByBlacklist</code>        | Chỉ blacklist một số đuôi nguy hiểm, không whitelist ảnh                                                               |
| <code>avatarDir.resolve(filename)</code> | Nếu <code>filename</code> chứa traversal/absolute path, path đích bị attacker ảnh hưởng                                |
| <code>transferTo(...)</code>             | Sink ghi file thật lên filesystem                                                                                      |
| <code>setAvatarUrl(...)</code>           | Lưu URL dựa trên filename không chuẩn hóa                                                                              |

## 4.2. Kiểm tra path construction

```
Path destination = avatarDir.resolve(filename);
```

Điểm cần đánh dấu khi review:

- Không gọi `normalize()` trên `destination` trước khi ghi file.
- Không kiểm tra `destination.startsWith(avatarDir)`.
- Không ép filename về basename bằng `Paths.get(filename).getFileName()`.
- Không tự sinh tên file server-side.
- Không giới hạn thư mục con hợp lệ dưới `avatars`.

Ví dụ logic fix đã được comment ngay dưới sink:

```
String extension = getSafeImageExtension(contentType);
String safeFilename = UUID.randomUUID() + extension;
Path destination = avatarDir.resolve(safeFilename).normalize();
if (!destination.startsWith(avatarDir)) {
    throw new IllegalArgumentException("Invalid upload path");
}

file.transferTo(destination.toFile());
user.setAvatarUrl("/uploads/avatars/" + safeFilename);
```

## 4.3. Truy ngược validation blacklist

```
private static final Set<String> BLOCKED_FILE_EXTENSIONS = Set.of(
    ".exe",
    ".bat",
    ".cmd",
    ".sh",
    ".jar",
    ".jsp",
    ".xml");

private boolean isBlockedByBlacklist(String filename) {
    return BLOCKED_FILE_EXTENSIONS.stream().anyMatch(filename::endsWith);
}
```

| Vấn đề                               | Vì sao nguy hiểm                                                                                                                                                   |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Blacklist thay vì whitelist          | File không nằm trong danh sách chặn vẫn được ghi, ví dụ <code>.html</code> , <code>.svg</code> , <code>.php</code> , <code>.phtml</code> , hoặc định dạng polyglot |
| <code>endsWith</code> case-sensitive | <code>.JSP</code> , <code>.Xml</code> , <code>.Sh</code> không khớp blacklist hiện tại                                                                             |
| Không kiểm tra MIME thật             | <code>file.getContentType()</code> không được dùng; magic bytes cũng không được kiểm tra                                                                           |
| Không tách extension cuối an toàn    | Tên kiểu nhiều đuôi có thể qua được chính sách nếu đuôi cuối không bị blacklist                                                                                    |
| Không đổi tên file                   | Attacker giữ nguyên tên file và cấu trúc path                                                                                                                      |

#### 4.4. Truy ngược validation path traversal

```
private static final Pattern OBVIOUS_PARENT_TRAVERSAL =
    Pattern.compile("(^|/)\.\./(?!/);

private void rejectRelativeParentPath(String value, String message) {
    if (OBVIOUS_PARENT_TRAVERSAL.matcher(value).find()) {
        throw new IllegalArgumentException(message);
    }
}
```

| Dấu hiệu                                           | Kết luận                                                                                      |
|----------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Regex chỉ nhìn dấu <code>/</code>                  | Không xử lý separator khác trong một số môi trường                                            |
| Pattern có <code>(?!/)</code> sau <code>../</code> | Chặn <code>../file</code> , nhưng có thể bỏ sót dạng separator lặp như <code>..///file</code> |
| Không normalize sau filter                         | Filesystem vẫn có thể resolve parent segment khi ghi                                          |
| Không so sánh với root thật                        | Không có invariant <code>finalPath.startsWith(avatarDir)</code>                               |

Comment fix hiện có trong code đã nêu đúng hướng:

```
private static final Pattern OBVIOUS_PARENT_TRAVERSAL =
    Pattern.compile("(^|[\\\\\\\\/])\\.\\.(:[\\\\\\\\/]+|)$");
```

Tuy nhiên, regex chỉ nên là lớp phụ. Fix chính vẫn là:

```

Path destination = avatarDir.resolve(safeFilename).normalize();
if (!destination.startsWith(avatarDir)) {
    throw new IllegalArgumentException("Invalid upload path");
}

```

#### 4.5. Truy ngược source filename

```

private String resolveUploadFilename(MultipartFile file, String
requestedFilename) {
    if (requestedFilename != null && !requestedFilename.isBlank()) {
        return requestedFilename.trim();           // [SOURCE-1] multipart field
"filename"
    }
    return file.getOriginalFilename();             // [SOURCE-2] client-
controlled original filename
}

```

| Source                                  | Điều khiển bởi ai                                   | Ghi chú                                               |
|-----------------------------------------|-----------------------------------------------------|-------------------------------------------------------|
| <code>requestedFilename</code>          | Client gửi multipart field<br><code>filename</code> | UI hiện tại không gửi, nhưng API<br>backend vẫn nhận  |
| <code>file.getOriginalFilename()</code> | Client multipart upload                             | Không đáng tin; có thể chứa tên lạ/path<br>tùy client |

#### 4.6. Truy ngược controller endpoint

**File:** `src/main/java/org/example/controller/UserController.java`

```

@PostMapping(value =("/{id}/avatar", consumes =
MediaType.MULTIPART_FORM_DATA_VALUE)
public ResponseEntity<?> uploadAvatar(@PathVariable Integer id,
@RequestParam("file") MultipartFile
file,
@RequestParam(value = "filename",
required = false) String filename,
Authentication authentication) {
    try {
        if (!canAccessUser(authentication, id)) {
            return ResponseEntity.status(403).body("Forbidden");
        }
    }
}

```

```

        User user = userService.uploadAvatar(id, file, filename);
        return ResponseEntity.ok().body(user);
    }
}

```

| Điểm đọc                                            | Kết luận                                                            |
|-----------------------------------------------------|---------------------------------------------------------------------|
| @RequestParam("file") MultipartFile file            | Source file content                                                 |
| @RequestParam(value = "filename", required = false) | Source filename trực tiếp, dù UI không dùng                         |
| canAccessUser(authentication, id)                   | Giới hạn user upload avatar của chính mình hoặc admin               |
| userService.uploadAvatar(...)                       | Data đi vào service sink không qua normalize/allowlist ở controller |

#### 4.7. Truy ngược public exposure

**File:** src/main/java/org/example/config/WebConfig.java

```

Path uploadPath = Paths.get(uploadDir).toAbsolutePath().normalize();
registry.addResourceHandler("/uploads/**")
        .addResourceLocations(uploadPath.toUri().toString());

```

**File:** src/main/java/org/example/security/WebSecurityConfig.java

```

.requestMatchers(
    "/v3/**",
    "/swagger-ui/**",
    "/swagger-ui",
    "/swagger-ui.html",
    "/uploads/**"
).permitAll()

```

| Ý nghĩa                            | Tác động                                             |
|------------------------------------|------------------------------------------------------|
| /uploads/** map tới app.upload-dir | File ghi dưới upload root có thể được truy cập HTTP  |
| /uploads/** được permitAll()       | Không cần JWT để xem file đã upload                  |
| avatarUrl lưu URL public           | User profile dùng lại đường dẫn này để render avatar |

## 5. Hướng 2 - Truy vết từ source đến sink

### 5.1. Source bình thường từ UI profile

**File:** frontend/src/pages/Profile.jsx

```
const handleAvatarChange = async (e) => {
  const file = e.target.files?.[0];          // [SOURCE] file từ input browser
  e.target.value = '';

  if (!file) {
    return;
  }
  if (file.size > 5 * 1024 * 1024) {          // [CLIENT CHECK] chỉ size, không
    // đủ bảo vệ server
    toast.error('Ảnh đại diện không được vượt quá 5MB');
    return;
  }

  const response = await uploadUserAvatar(user.id, file);
};

<input
  type="file"
  className="hidden"
  onChange={handleAvatarChange}
  disabled={avatarUploading}
/>
```

| Điểm đọc                               | Kết luận                                                                                                      |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>type="file"</code>               | Người dùng chọn file ở browser                                                                                |
| Check size ở client                    | Có thể bypass bằng request trực tiếp; server chỉ dựa vào multipart max-size                                   |
| Không có <code>accept="image/*"</code> | UI không tự hạn chế ảnh, nhưng kể cả có cũng không phải boundary bảo mật                                      |
| Không gửi <code>filename</code>        | Flow bình thường dùng <code>OriginalFilename</code> ; attacker vẫn có thể tự thêm field <code>filename</code> |

### 5.2. Source đi qua API client



**File:** frontend/src/services/api.js

```
export const uploadUserAvatar = (userId, file) => {
  const formData = new FormData();
  formData.append('file', file); // [SOURCE] multipart part "file"

  return api.post(`/user/${userId}/avatar`, formData);
};
```

| Điểm đọc                      | Kết luận                                             |
|-------------------------------|------------------------------------------------------|
| FormData.append('file', file) | Browser gửi filename mặc định của file               |
| Không set filename field      | UI sạch, nhưng backend vẫn expose parameter optional |
| api tự gắn JWT                | Endpoint yêu cầu auth                                |

### 5.3. Source tới controller binding

```
@RequestParam("file") MultipartFile file,
@RequestParam(value = "filename", required = false) String filename
```

Tại đây có hai nguồn input:

1. file : nội dung file và original filename.
2. filename : field multipart tùy ý nếu attacker gửi request thủ công.

### 5.4. Controller chuyển nguyên input xuống service

```
User user = userService.uploadAvatar(id, file, filename);
```

Không có bước nào ở controller để:

- ép filename về basename;
- whitelist MIME/extension;
- sinh tên file mới;
- normalize path;

- chặn absolute path;
- kiểm tra final path còn nằm trong `avatarDir`.

## 5.5. Service nhận source và quyết định filename

```
String filename = resolveUploadFilename(file, requestedFilename);
```

Luồng rẽ nhánh:

| Nhánh                          | Điều kiện                                           | Filename dùng để ghi                    |
|--------------------------------|-----------------------------------------------------|-----------------------------------------|
| <code>requestedFilename</code> | Multipart có field <code>filename</code> không rỗng | <code>requestedFilename.trim()</code>   |
| <code>OriginalFilename</code>  | Không có field <code>filename</code>                | <code>file.getOriginalFilename()</code> |

## 5.6. Validation không cắt được dữ liệu nguy hiểm

```
rejectRelativeParentPath(filename, "Filename cannot contain ../");
if (isBlockedByBlacklist(filename)) {
    throw new IllegalArgumentException("This file type is not allowed");
}
```

| Dạng kiểm tra           | Có                                                                                                                                                    | Thiếu                                                                      |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Chặn một vài extension  | Có blacklist <code>.exe</code> , <code>.bat</code> , <code>.cmd</code> , <code>.sh</code> , <code>.jar</code> , <code>.jsp</code> , <code>.xml</code> | Không whitelist ảnh                                                        |
| Chặn traversal đơn giản | Có regex cho một số dạng <code>../</code>                                                                                                             | Không normalize, không <code>startsWith</code> , bypass được separator lặp |
| Chặn content độc hại    | Không                                                                                                                                                 | Không kiểm tra MIME thật/magic bytes                                       |
| Chặn filename đặc biệt  | Không đầy đủ                                                                                                                                          | Không reject absolute path, control chars, reserved names                  |

## 5.7. Input chạm sink

```
Path destination = avatarDir.resolve(filename);
file.transferTo(destination.toFile());
```

Từ góc nhìn source-to-sink, đây là điểm xác nhận lỗ hổng:

- Source `filename` vẫn giữ quyền điều khiển path.
- Sink `transferTo` nhận `destination` bị ảnh hưởng bởi source.
- Không có sanitizer mạnh nằm giữa source và sink.

## 5.8. Output được lưu và public hóa

```
user.setAvatarUrl("/uploads/avatars/" + filename);
return userRepository.save(user);
```

Sau ghi file:

- DB lưu `avatarUrl`.
- Profile render avatar bằng URL đó.
- Static route `/uploads/**` cho phép truy cập public nếu file nằm dưới `app.upload-dir`.

---

## 6. Hai biến thể phát hiện chính

### 6.1. Blacklist bypass

| Bước review                   | Dấu hiệu                                                                                     |
|-------------------------------|----------------------------------------------------------------------------------------------|
| Tìm extension policy          | <code>BLOCKED_FILE_EXTENSIONS</code> là blacklist                                            |
| Tìm cách so sánh              | <code>filename::endsWith</code> , case-sensitive                                             |
| Tìm allowlist ảnh             | Không có <code>ALLOWED_IMAGE_TYPES</code> , không có <code>getSafeImageExtension</code> thật |
| Tìm server-generated filename | Không có <code>UUID.randomUUID()</code> trong code chạy thật                                 |
| Tìm content validation        | Không kiểm tra MIME/magic bytes                                                              |

Kết luận phát hiện:

Nếu chỉ blacklist một số đuôi và vẫn cho client quyết định filename, thì reviewer đánh dấu FileUpload blacklist bypass.

## 6.2. Path traversal

| Bước review            | Dấu hiệu                                                                                                      |
|------------------------|---------------------------------------------------------------------------------------------------------------|
| Tìm nơi ghép path      | <code>avatarDir.resolve(filename)</code>                                                                      |
| Tìm sanitizer          | Regex <code>OBVIOUS_PARENT_TRAVERSAL</code>                                                                   |
| Kiểm tra normalize     | Không có <code>destination.normalize()</code> trước <code>transferTo</code>                                   |
| Kiểm tra boundary      | Không có <code>destination.startsWith(avatarDir)</code>                                                       |
| Kiểm tra absolute path | Không reject filename absolute                                                                                |
| Kiểm tra parent mkdir  | <code>Files.createDirectories(destination.getParent())</code> có thể chuẩn bị thư mục theo path attacker chọn |

Kết luận phát hiện:

Nếu attacker-controlled filename đi vào `Path.resolve(...)` rồi `transferTo(...)` mà không có `normalize + startsWith(baseDir)`, đánh dấu arbitrary file write/path traversal.

## 7. Checklist tái rà soát FileUpload

- ✓ Tìm tất cả `MultipartFile`, `transferTo`, `Files.write`, `Files.copy`.
- ✓ Xác định source: `file`, `filename`, `getOriginalFilename()`.
- ✓ Xác định sink: `file.transferTo(destination.toFile())`.
- ✓ Kiểm tra path building: `avatarDir.resolve(filename)`.
- ✓ Kiểm tra sanitizer: regex yếu, không normalize boundary.

- ✓ Kiểm tra file type policy: blacklist thay vì whitelist.
- ✓ Kiểm tra public serving: `/uploads/**` `permitAll`.
- ✓ Kiểm tra stored output: `user.avatarUrl`.
- ✓ Đối chiếu comment fix code ngay dưới sink.

## 8. Code fix tham chiếu từ comment hiện có

Code comment hiện tại trong `UserServiceImpl` đã nêu hướng fix đúng. Khi chuyển sang code thật, nên gom thành các invariant bắt buộc:

```
String contentType = file.getContentType();
if (contentType == null ||
    !ALLOWED_IMAGE_TYPES.contains(contentType.toLowerCase(Locale.ROOT)))
{
    throw new IllegalArgumentException("Only images are allowed");
}

String extension = getSafeImageExtension(contentType);
String safeFilename = UUID.randomUUID() + extension;

Path avatarDir = Paths.get(uploadDir,
    "avatars").toAbsolutePath().normalize();
Path destination = avatarDir.resolve(safeFilename).normalize();
if (!destination.startsWith(avatarDir)) {
    throw new IllegalArgumentException("Invalid upload path");
}

file.transferTo(destination.toFile());
user.setAvatarUrl("/uploads/avatars/" + safeFilename);
```

Checklist fix tối thiểu:

- ☐ Không dùng `requestedFilename` hoặc `getOriginalFilename()` làm stored filename.
- ☐ Dùng whitelist MIME/extension ảnh.
- ☐ Sinh filename server-side bằng UUID/random.
- ☐ Normalize final path.
- ☐ Bắt buộc final path nằm trong `avatarDir`.
- ☐ Giới hạn kích thước ở server.

- Cân nhắc strip metadata và re-encode ảnh nếu đây là production flow.

## 9. File và dòng quan trọng

| File                                                                    | Dòng      | Vai trò                                                                            |
|-------------------------------------------------------------------------|-----------|------------------------------------------------------------------------------------|
| <code>frontend/src/pages/Profile.jsx</code>                             | 230-263   | UI source: chọn file và gọi upload                                                 |
| <code>frontend/src/services/api.js</code>                               | 51-56     | API source: tạo multipart form                                                     |
| <code>src/main/java/org/example/controller/UserController.java</code>   | 142-158   | Backend source: bind multipart <code>file</code> và optional <code>filename</code> |
| <code>src/main/java/org/example/serviceImpl/UserServiceImpl.java</code> | 32-40     | Blacklist extension + regex traversal yếu                                          |
| <code>src/main/java/org/example/serviceImpl/UserServiceImpl.java</code> | 98-151    | Core vulnerable upload flow                                                        |
| <code>src/main/java/org/example/serviceImpl/UserServiceImpl.java</code> | 154-168   | Helper blacklist/source filename/traversal filter                                  |
| <code>src/main/java/org/example/config/WebConfig.java</code>            | 31-35     | Static handler <code>/uploads/**</code>                                            |
| <code>src/main/java/org/example/security/WebSecurityConfig.java</code>  | 81-96     | Security permit upload read + require auth for avatar upload                       |
| <code>src/main/resources/application.properties</code>                  | 17, 27-28 | Upload dir + multipart max size                                                    |
| <code>docker-compose.yml</code>                                         | 39, 52    | Docker upload dir <code>/app/uploads</code> mounted to <code>./uploads</code>      |

## 10. Final reviewer note

FileUpload flow này không chỉ là "cho upload file sai đuôi". Điểm nguy hiểm nằm ở tổ hợp:

1. client kiểm soát filename;
2. blacklist yếu;
3. traversal filter yếu;

4. `Path.resolve(filename)` không kiểm tra boundary;
5. `transferTo` ghi file thật;
6. `/uploads/**` public.

Vì vậy khi trình bày trong báo cáo whitebox, nên đánh dấu sink tại `transferTo(...)`, source tại `@RequestParam filename` / `getOriginalFilename()`, và invariant bị thiếu là:

```
finalPath = avatarDir.resolve(serverGeneratedSafeName).normalize()  
finalPath must start with avatarDir
```